ĐẠI HỌC QUỐC GIA HÀ NỘI

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO BÀI TẬP LỚN

ĐẶC TẢ NGÔN NGỮ LẬP TRÌNH

NGUYÊN LÝ NGÔN NGỮ LẬP TRÌNH

Giảng viên: TS. Võ Đình Hiếu

Nhóm:
Hồ Thu Giang (25025060)
Nguyễn Thị Ngọc Minh (25025080)

Năm 2026

1. Giới thiệu

Ngôn ngữ GMLang được xây dựng dựa trên ngôn ngữ UPL trong đề bài, với mục tiêu mở rộng khả năng biểu đạt và hỗ trợ tốt hơn cho việc xây dựng trình biên dịch.

GMLang vẫn giữ các đặc điểm cốt lõi của UPL như cú pháp đơn giản, số lượng kiểu dữ liệu hạn chế và các cấu trúc điều khiển cơ bản, nhưng được cải tiến thêm một số tính năng như:

- Suy luận kiểu (type inference)
- Biểu thức logic (&&, ||, !)
- Sinh Java Bytecode để thực thi trên máy ảo Java (JVM)

Mục tiêu của bài là xây dựng một pipeline biên dịch đầy đủ từ mã nguồn GMLang đến bytecode, giúp minh họa rõ ràng cách hoạt động của một trình biên dịch.

---

2. Mục tiêu

Thiết kế ngôn ngữ GMLang với cú pháp và ngữ nghĩa rõ ràng

Xây dựng các thành phần của trình biên dịch:

- Phân tích từ vựng (Lexer)
- Phân tích cú pháp (Parser)
- Phân tích ngữ nghĩa (Semantic Analysis)
- Sinh bytecode (Code Generation) và thực thi trên máy ảo Java (JVM)

---

3. Đặc tả từ vựng (Lexical Specification)

Ngôn ngữ GMLang bao gồm các loại token chính như sau:

3.1. Từ khóa (Keywords)

Các từ khóa được định nghĩa trước và không thể sử dụng làm tên biến, bao gồm:

```
begin, end, int, bool, float, string,
auto, if, then, else, elseif, do,
while, for, to, print, input
```

3.2. Định danh (Identifier)

```
IDENTIFIER = [a-zA-Z]+[0-9]*
```

Định danh được dùng để đặt tên biến và phải tuân theo quy tắc:

- Bắt đầu bằng ký tự chữ cái
- Có thể chứa các chữ cái tiếp theo
- Nếu có chữ số, chỉ được xuất hiện ở cuối

Ví dụ hợp lệ:

```
x, abc, var123
```

Ví dụ không hợp lệ:

```
1abc, ab1c
```

3.3. Hằng số (Literals)

Ngôn ngữ hỗ trợ hai loại hằng:

- Số nguyên: 0, 1, 100,...
- Giá trị logic: true, false
- Số thập phân: 0.1, 0.02,...
- Chuỗi: abc, xyz,...

3.4. Toán tử (Operators)

GMLang hỗ trợ các toán tử:

- Toán học: +, *, -, /
- So sánh: >, >=, ==, <=, <
- Logic: &&, ||, !
- Gán: =

3.5. Ký hiệu phân tách (Separators)

Các ký hiệu phân tách gồm:

```
; ( ) { }
```

3.6. Chú thích (Comments)

Ngôn ngữ hỗ trợ hai dạng chú thích:

```
// chú thích một dòng

/* chú thích nhiều dòng */
```

---

4. Đặc tả cú pháp (Syntax Specification)

Cấu trúc tổng quát chương trình

```
begin
{
  <khai báo và câu lệnh>
}
end
```

4.1. Khai báo biến

Biến phải được khai báo trước khi sử dụng. GMLang hỗ trợ ba kiểu khai báo:

- Khai báo thông thường:

```
int x;
bool a;
```

- Khai báo có khởi tạo:

```
int y = 5;
```

- Khai báo sử dụng suy luận kiểu:

```
auto z = 10;
auto b = true;
```

4.2. Câu lệnh gán

Câu lệnh gán có dạng:

```
x = 10;
y = x + 2;
```

4.3. Câu lệnh điều kiện

GMLang hỗ trợ ba dạng câu lệnh điều kiện:

- If - then

```
if (x > 5) then {
  print(x);
}
```

- If - then - else:

```
if (x > 5) then {
  print(x);
} else {
  print(0);
}
```

- If - then - elseif (có thể nhiều lần) - else:

```
if (x > 5) then {
   print(x);
} elseif (x > 3) {
   print(1);
} else {
   print (2);
}
```

GMLang hỗ trợ thêm cấu trúc elseif nhằm biểu diễn nhiều nhánh điều kiện một cách ngắn gọn. Về bản chất, elseif được chuyển đổi nội bộ thành các câu lệnh if lồng nhau trong quá trình sinh

AST.

4.4. Vòng lặp

Ngôn ngữ hỗ trợ hai loại vòng lặp:

Do – while

```
do {
  x = x + 1;
} while (x > 5);
```

For

```
for (x = 0 to 10 step 1) {
  print(x);
}
```

Vòng lặp `for` có dạng:

```
for (x = start to end step stepValue) {
  <statement>
}
```

Trong đó `x`, `start`, `end` và `stepValue` đều phải có kiểu `int`.

Cách thực thi:

- Gán giá trị ban đầu `start` cho biến điều khiển `x`.
- Nếu `stepValue > 0`, vòng lặp tiếp tục khi `x <= end`.
- Nếu `stepValue < 0`, vòng lặp tiếp tục khi `x >= end`.
- Sau mỗi lần lặp, biến điều khiển được cập nhật: `x = x + stepValue`.
- `stepValue = 0` là lỗi ngữ nghĩa vì có thể gây vòng lặp vô hạn.

Ví dụ vòng lặp tăng:

```
for (x = 0 to 3 step 1) {
  print(x);
}
```

Kết quả:

```
0
1
2
3
```

Ví dụ vòng lặp giảm:

```
for (x = 3 to 0 step -1) {
  print(x);
}
```

Kết quả:

```
3
2
1
0
```

Nếu hướng đi của `stepValue` không phù hợp với `start` và `end`, vòng lặp không thực hiện lần nào. Ví dụ:

```
for (x = 0 to 10 step -1) {
  print(x);
}
```

Trong trường hợp này, `stepValue < 0` nên điều kiện lặp là `x >= end`. Ban đầu `0 >= 10` là sai, vì vậy vòng lặp kết thúc ngay.

4.5. Lệnh in

Lệnh in có dạng:

```
print(x);
print(x + 1);
```

4.6. Biểu thức

GMLang hỗ trợ các loại biểu thức: - Biểu thức số học: +, -, *, / - Biểu thức so sánh: >, >=, ==, <= - Biểu thức logic: &&, ||, !

Ví dụ:

```
(x + 2) * 3 > 10 && !a
```

4.7. Độ ưu tiên toán tử

Mức	Toán tử
1	!
2	*, /
3	+, -
4	>, >=, ==, <=, <
5	&&
6	||

Dấu ngoặc () được sử dụng để thay đổi thứ tự ưu tiên của các phép toán.

Biểu thức nằm trong dấu ngoặc sẽ được tính trước, bất kể thứ tự ưu tiên mặc định của toán tử.

---

5. Văn phạm (Grammar – BNF)

```
Program ::= "begin" Block "end"

Block ::= "{" DeclList StmtList "}"

DeclList ::= Decl DeclList | epsilon
StmtList ::= Stmt StmtList | epsilon

Decl ::= Type Identifier ("=" Expr)? ";"

Type ::= "int" | "bool" | "auto" | "float" | "string"

InputStmt ::= "input" "(" Identifier ")" ";"

Stmt ::= AssignStmt
       | IfStmt
       | WhileStmt
       | ForStmt
       | PrintStmt
       | InputStmt
       | Block

AssignStmt ::= Identifier "=" Expr ";"
AssignExpr ::= Identifier "=" Expr

IfStmt ::= "if" "(" Expr ")" "then" Stmt IfTail

IfTail ::= "elseif" "(" Expr ")" Stmt ElseIfList "else" Stmt | "else" Stmt | epsilon

ElseIfList ::= "elseif" "(" Expr ")" Stmt ElseIfList | epsilon

WhileStmt ::= "do" Stmt "while" "(" Expr ")" ";"

ForStmt ::= "for" "(" Identifier "=" Expr "to" Expr "step" Expr ")" Stmt

PrintStmt ::= "print" "(" Expr ")" ";"

Expr ::= Expr "||" AndExpr | AndExpr
AndExpr ::= AndExpr "&&" EqualityExpr | EqualityExpr
EqualityExpr ::= EqualityExpr "==" RelExpr | RelExpr
RelExpr ::= RelExpr (">" | ">=" | "<" | "<=") AddExpr | AddExpr
AddExpr ::= AddExpr ("+" | "-") MulExpr | MulExpr
MulExpr ::= MulExpr ("*" | "/") UnaryExpr | UnaryExpr
UnaryExpr ::= "!" UnaryExpr | "-" UnaryExpr | Primary

Literal ::= INT_LITERAL | BOOL_LITERAL | FLOAT_LITERAL | STRING_LITERAL
Primary ::= Identifier | Literal | "(" Expr ")"
```

---

6. Phân tích cú pháp (Parser & AST)

Sử dụng **Recursive Descent Parser**.

Ví dụ AST:

```
y = x + 2

Assign
|-- id: y
`-- Add
    |-- x
    `-- 2
```

---

7. Đặc tả ngữ nghĩa (Semantic Specification)

7.1. Bảng ký hiệu (Symbol Table)

Bảng ký hiệu (Symbol Table) được sử dụng trong quá trình phân tích ngữ nghĩa để lưu trữ thông tin về các biến đã được khai báo trong chương trình.

Mỗi mục trong bảng ký hiệu bao gồm các thông tin sau:

Tên biến	Kiểu	Phạm vi	Index
x	int	global	1
y	int	block	2

Trong đó: - Tên biến (Name): định danh của biến - Kiểu dữ liệu (Type): int, bool, string, float (hoặc suy luận từ auto) - Phạm vi (Scope): xác định theo khối { } - Chỉ số (Index): vị trí của biến trong local variable của JVM

Cách hoạt động

- Khi khai báo:
  - Thêm vào Symbol Table
  - Kiểm tra trùng tên
- Khi sử dụng:
  - Tra cứu bảng
  - Kiểm tra kiểu
- Khi ra khỏi block:
  - Xóa biến khỏi bảng

Ánh xạ JVM

Mỗi biến sẽ được ánh xạ sang một local variable slot

```
x → slot 1 → istore_1 / iload_1
y → slot 2 → istore_2 / iload_2
```

7.2. Phạm vi (Scope)

- Phạm vi của biến được xác định bởi các khối `{ }`
- Biến chỉ có hiệu lực trong phạm vi mà nó được khai báo và các phạm vi con

7.3. Kiểm tra khai báo

- Biến phải được khai báo trước khi sử dụng
- Không được khai báo trùng tên trong cùng một phạm vi.

7.4. Kiểm tra kiểu dữ liệu

Các quy tắc kiểm tra kiểu:

- Phép +, -, *, / chỉ áp dụng cho kiểu int, float
- Phép so sánh >, >=, <, <= chỉ áp dụng cho int, float
- Phép == áp dụng cho cả int, bool, float, string
- Với kiểu string, phép == so sánh nội dung chuỗi.
- Các phép logic (&&, ||, !) chỉ áp dụng cho bool.
- Trong vòng lặp for, biến điều khiển, start, end và step phải có kiểu int; step không được bằng 0.

7.5. Suy luận kiểu (Type Inference)

Từ khóa auto cho phép trình biên dịch tự động suy ra kiểu của biến dựa trên giá trị khởi tạo.

```
auto x = 5;          // int
auto a = true;       // bool
auto b = 5.5;         // float
auto c = "hello";     // string
```

7.6. Lỗi ngữ nghĩa

Một số lỗi phổ biến:

- Gán sai kiểu:

```
int x;
x = true;    // sai kiểu
```

- Sử dụng biến chưa được khai báo

```
x = 5;       // chưa khai báo
```

---

8. Sinh mã Bytecode

Trình biên dịch GMLang sinh Bytecode. Mỗi câu lệnh trong GMLang được ánh xạ trực tiếp sang các lệnh tương ứng trong JVM.

Một số lệnh cơ bản:

- Nạp hằng số: iconst_0, iconst_1, ldc
- Truy cập biến: iload, istore
- Toán học: iadd, isub, imul, idiv
- So sánh: if_icmpgt, if_icmpge, if_icmplt, if_icmple, if_icmpeq
- Điều khiển luồng: goto, ifeq
- In: getstatic java/lang/System/out Ljava/io/PrintStream;

  invokevirtual java/io/PrintStream/println(I)V

Chương trình sau khi biên dịch sẽ được thực thi trực tiếp trên JVM

---

9. Xử lý lỗi (Error Handling)

Trong GMLang, xử lý lỗi là một phần quan trọng của quá trình biên dịch. Mục tiêu không chỉ là phát hiện chương trình sai, mà còn cung cấp thông tin đủ rõ để người lập trình biết lỗi nằm ở đâu và thuộc loại nào.

Trình biên dịch phát hiện ba nhóm lỗi chính:

Loại lỗi	Giai đoạn phát hiện	Ví dụ
Lỗi từ vựng	Lexer	ký tự không hợp lệ, token không thuộc ngôn ngữ
Lỗi cú pháp	Parser	thiếu `;`, thiếu `}`, sai cấu trúc `if`, `for`
Lỗi ngữ nghĩa	Semantic Analyzer	sai kiểu, biến chưa khai báo, khai báo trùng

9.1. Lỗi từ vựng

Lỗi từ vựng xảy ra khi mã nguồn chứa ký tự hoặc chuỗi ký tự không thể tạo thành token hợp lệ của GMLang.

Ví dụ:

```
int x = 10 @;
```

Ký tự `@` không thuộc tập toán tử hoặc ký hiệu phân tách của GMLang, vì vậy lexer báo lỗi.

Đặc điểm:

- Lỗi được báo kèm vị trí dòng/cột.

- Lexer thường dừng tại lỗi đầu tiên vì nếu token không hợp lệ thì parser không có đầu vào đáng tin cậy để tiếp tục.

9.2. Lỗi cú pháp

Lỗi cú pháp xảy ra khi chuỗi token không khớp với grammar của ngôn ngữ.

Ví dụ:

```
begin
{
  int x = ;
  print(x)
}
end
```

Các lỗi trong ví dụ:

- Thiếu biểu thức sau dấu `=`.
- Thiếu dấu `;` sau lệnh `print(x)`.

Chiến lược xử lý lỗi cú pháp:

- Parser ghi nhận lỗi tại vị trí phát hiện.
- Sau khi gặp lỗi, parser thực hiện recovery bằng cách bỏ qua token cho đến khi gặp một điểm đồng bộ.
- Các điểm đồng bộ chính là `;` và `}`.
- Sau đó parser tiếp tục phân tích phần còn lại của chương trình.

Nhờ đó, parser có thể báo nhiều lỗi cú pháp trong một lần biên dịch thay vì chỉ dừng ở lỗi đầu tiên.

Lưu ý: GMLang quy định trong một block, phần khai báo phải đứng trước phần câu lệnh:

```
Block ::= "{" DeclList StmtList "}"
```

Vì vậy đoạn sau là lỗi cú pháp:

```
begin
{
  int x;
  x = 1;
  int y;
}
end
```

Lý do là `int y;` xuất hiện sau một câu lệnh gán, trong khi grammar không cho phép quay lại phần khai báo sau khi đã bắt đầu phần câu lệnh.

9.3. Lỗi ngữ nghĩa

Lỗi ngữ nghĩa xảy ra khi chương trình đúng cú pháp nhưng vi phạm các luật về kiểu dữ liệu, khai báo hoặc phạm vi.

Ví dụ:

```
begin
{
  int x;
  bool flag = true;

  x = true;
  y = 10;
  if (x) then {
    print(x);
  }
}
end
```

Các lỗi ngữ nghĩa:

- `x = true;`: không thể gán giá trị `bool` cho biến `int`.
- `y = 10;`: biến `y` chưa được khai báo.
- `if (x)`: điều kiện `if` phải có kiểu `bool`, nhưng `x` có kiểu `int`.

Semantic analyzer kiểm tra các luật chính:

- Biến phải được khai báo trước khi sử dụng.
- Không được khai báo trùng tên trong cùng một scope.
- Kiểu của biểu thức gán phải tương thích với kiểu của biến.
- Biến `auto` phải có giá trị khởi tạo để suy luận kiểu.
- Điều kiện trong `if`, `elseif`, `while` phải có kiểu `bool`.
- Trong vòng `for`, biến điều khiển, `start`, `end`, `step` phải có kiểu `int`.
- `step` của vòng `for` không được bằng `0`.

Chiến lược xử lý lỗi ngữ nghĩa:

- Semantic analyzer không dừng ngay tại lỗi đầu tiên.
- Khi phát hiện lỗi trong một khai báo hoặc câu lệnh, lỗi được lưu lại.
- Analyzer tiếp tục kiểm tra các khai báo/câu lệnh còn lại trong block nếu có thể.
- Sau khi duyệt xong, toàn bộ danh sách lỗi được báo ra cùng lúc.

9.4. Định dạng thông báo lỗi

Mỗi lỗi cần cung cấp đủ thông tin để người dùng xác định nguyên nhân:

```
<loại lỗi>: <mô tả lỗi> at <dòng>:<cột>
```

Ví dụ:

```
Cannot assign bool to int at 7:5
Undefined variable 'y' at 8:5
Condition must be bool at 10:9
```

Khi có nhiều lỗi, trình biên dịch gom các lỗi thành một danh sách:

```
error: Semantic analysis failed with 3 error(s):
Cannot assign bool to int at 7:5
```

```
Undefined variable 'y' at 8:5
Condition must be bool at 10:9
```

9.5. Ảnh hưởng của lỗi đến quá trình biên dịch

Quy trình biên dịch chỉ tiếp tục sang pha tiếp theo khi pha hiện tại thành công.

```
Lexer
  -> Parser
  -> Semantic Analyzer
  -> Code Generator
  -> Class File Writer
```

Nếu một pha phát hiện lỗi nghiêm trọng:

- Lexer lỗi: dừng trước parser.
- Parser lỗi: dừng trước semantic analyzer.
- Semantic lỗi: dừng trước code generation.
- Code generation lỗi: không tạo file `.class` hợp lệ.

Như vậy, file bytecode chỉ được sinh khi chương trình vượt qua đầy đủ các bước kiểm tra từ vựng, cú pháp và ngữ nghĩa.

---

10. Quy trình xây dựng trình biên dịch và điểm cải tiến

Ngôn ngữ GMLang không chỉ dừng lại ở việc định nghĩa cú pháp và ngữ nghĩa, mà còn được thiết kế để hỗ trợ một quy trình biên dịch hoàn chỉnh từ mã nguồn đến bytecode.

Quá trình biên dịch của GMLang được tổ chức theo các bước liên tiếp như sau:

- Đầu tiên, bộ phân tích từ vựng (Lexer) thực hiện việc đọc mã nguồn và chuyển đổi chuỗi ký tự thành các token. Giai đoạn này chịu trách nhiệm loại bỏ khoảng trắng, nhận diện từ khóa, định danh, toán tử và phát hiện các lỗi từ vựng.
- Tiếp theo, bộ phân tích cú pháp (Parser) sử dụng các token để xây dựng cây cú pháp trừu tượng (AST). AST biểu diễn cấu trúc chương trình dưới dạng cây, giúp dễ dàng xử lý ở các bước sau.
- Sau đó, bộ phân tích ngữ nghĩa (Semantic Analyzer) kiểm tra tính đúng đắn của chương trình, bao gồm kiểm tra khai báo biến, kiểm tra kiểu dữ liệu và xử lý phạm vi. Đồng thời, trong GMLang, giai đoạn này còn thực hiện suy luận kiểu cho các biến được khai báo bằng từ khóa auto.
- Sau khi hoàn thành phân tích ngữ nghĩa, trình biên dịch tiến hành sinh trực tiếp Java Bytecode từ cây AST. Mỗi node trong AST sẽ được duyệt và ánh xạ sang các lệnh tương ứng của JVM.

```
Source Code (GMLang)
↓
Lexical Analysis
↓
Syntax Analysis (AST)
```

```
↓
Semantic Analysis
↓
Java Bytecode Generation
↓
Java Bytecode (JVM)
```

Bên cạnh việc xây dựng đầy đủ pipeline compiler, GMLang còn có một số điểm cải tiến so với ngôn ngữ ban đầu:

- Thứ nhất, ngôn ngữ bổ sung cơ chế suy luận kiểu thông qua từ khóa auto, giúp giảm bớt việc khai báo kiểu và tăng tính linh hoạt.
- Thứ hai, hệ thống biểu thức được mở rộng với các toán tử logic như &&, ||, !, cho phép biểu diễn các điều kiện phức tạp hơn.
- Việc sinh trực tiếp Java Bytecode giúp giảm độ phức tạp của hệ thống và tận dụng trực tiếp môi trường thực thi của JVM.
- Cuối cùng, trình biên dịch được xây dựng theo mô hình nhiều pha, tương tự các hệ thống compiler thực tế, giúp tăng tính rõ ràng, dễ bảo trì và dễ mở rộng.

---

11. Ví dụ full pipeline

Source

```
int x = 1 + 2;
```

Token

```
int, x, =, 1, +, 2
```

AST

```
Assign
|-- id: x
`-- Add
    |-- 1
    `-- 2
```

JVM

```
iconst_1
iconst_2
iadd
istore_1
```

---

12. Kết luận

GMLang là phiên bản cải tiến của UPL với cú pháp đơn giản nhưng mạnh hơn, hỗ trợ biểu thức logic, suy luận kiểu và có thiết kế trình biên dịch đầy đủ. Ngôn ngữ này không chỉ đáp ứng yêu cầu của đề bài mà còn thể hiện rõ cấu trúc của một hệ thống compiler hiện đại, dễ mở rộng và tối ưu trong tương lai.

GMLang có thể mở rộng để hỗ trợ:

- function
- array
- optimization (constant folding)